

Gyroid-Sphere: Open-Surface Construction by Spherical Clipping, with a Two-Sided Lighting Pitfall

engvis

June 19, 2026

1 Goal

Render the gyroid implicit surface

$$f(x, y, z) = \sin x \cos y + \sin y \cos z + \sin z \cos x = 0$$

restricted to the open ball $B = \{\mathbf{p} : \|\mathbf{p}\| < R\}$ with $R = 1$. The result is a single open surface patch

$$S = \{f = 0\} \cap B,$$

not a closed solid: only the gyroid two-manifold inside the ball, with its boundary lying exactly on the sphere ∂B .

2 Why Boolean SDF Combinations Are Wrong

A first instinct is to combine the gyroid SDF g and the ball SDF $s = \|\mathbf{p}\| - R$ with a Boolean op:

Operator	Set produced	What we get
$\min(g, s)$	union	gyroid + ball envelope
$\max(g, s)$	intersection	gyroid “capped” by ball faces

Both produce a *closed solid* (gyroid skin plus a spherical cap), not a thin two-manifold. This is the wrong topology: the iso-surface $\{g = 0, s \leq 0\}$ is a 2-manifold with boundary, while $\{\max(g, s) = 0\}$ is the boundary of a 3-manifold and contains spurious “cap” regions where $g > 0$.

3 Construction: Pure Gyroid Mesh + Spherical Mesh Clipping

The chosen pipeline (Figure 1) keeps g pure and clips the resulting triangle mesh:

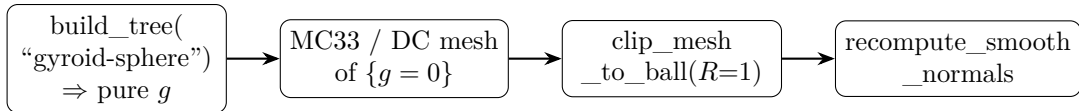


Figure 1: gyroid-sphere construction. The implicit tree is the pure gyroid; the ball is enforced as a post-processing step on the triangle mesh.

3.1 Exact spherical cutting

For each triangle (V_0, V_1, V_2) let $d_i = \|V_i - c\|^2 - R^2$. An inside vertex satisfies $d_i \leq 0$. We classify the triangle by the number k of inside vertices:

- $k = 3$: keep verbatim.
- $k = 0$: discard.
- $k = 1$: split into one new triangle (V_{in}, A, B) where A, B are the sphere-edge intersections.
- $k = 2$: split into two triangles $(V_{\text{next}}, V_{\text{prev}}, A)$ and (V_{next}, A, B) to fill the remaining quad.

The intersection on edge (V_a, V_b) is solved analytically. Writing $P(t) = V_a + t(V_b - V_a)$ and substituting into $\|P - c\|^2 = R^2$ gives a quadratic; the inside–outside transition gives the linear approximation

$$t \approx \frac{d_a}{d_a - d_b},$$

which is sufficient since $|d_a| + |d_b|$ is small relative to R^2 at mesh resolution. The resulting boundary vertices lie on ∂B to numerical precision.

3.2 Why we recompute smooth normals after clipping

The clip introduces new vertices with zero normals. We initially tried to overwrite all normals using the analytic gradient ∇g via `overwrite_normals_with_gradient`. Empirically the `fidget` grad evaluator returned $(0, 0, 0)$ for these points; the function silently keeps the old normal. For untouched gyroid vertices this was harmless because `from_triangles_open` had already filled in smooth normals. For the new clip vertices it left the normals at zero — causing the “black surface” bug (Section 4).

The fix is the standalone helper `recompute_smooth_normals(&mut mesh)`, which computes area-weighted vertex normals from triangle geometry alone:

$$\mathbf{n}_v = \frac{\sum_{T \ni v} (\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)}{\|\sum_T (\cdots)\|}.$$

This depends only on the mesh and so works after the clip.

4 Black-Surface Bug, Three Layers Deep

After the clip pipeline was complete, the gyroid-sphere patch rendered *very dark*, while gyroid (no clip) looked correct. Three distinct issues conspired:

4.1 Layer 1: Open back face never lit

After the clip the mesh is open: the back side of the gyroid sheet now faces the camera at oblique angles. Standard back-face culling (`cullMode = Back`) drops half the triangles. We turned off back-face culling, but with $\cos \theta = \mathbf{N} \cdot \mathbf{L}$ clamped to ≥ 0 , the back-facing fragments returned $\cos \theta = 0$ and thus zero diffuse, leaving them lit only by ambient.

The standard remedy is two-sided lighting: in the fragment shader, if the geometric normal faces away from the camera, flip it.

```
let V = normalize(scene.camera_pos - in.world_position);
if (dot(N, V) < 0.0) { N = -N; }
```

4.2 Layer 2: NaN propagation from a zero tangent

Even after enabling two-sided lighting, the patch was still very dark. Inspection of one fragment showed $L_{\text{out}} \approx \text{ambient}$, exactly as if $\cos \theta \equiv 0$.

The root cause was the *tangent*, not the normal. Newly created clip vertices were initialised with `tangent = [0.0, 0.0, 0.0, 1.0]`. The shader does

```
let T = normalize(in.world_tangent); // normalize([0,0,0]) = NaN
let TBN = mat3x3<f32>(T, B, Ng);
var N = normalize(TBN * sampled_normal);
```

`normalize([0,0,0])` is NaN; the TBN matrix is poisoned; N is NaN; $\max(\mathbf{N} \cdot \mathbf{L}, 0) = 0$ drops the entire diffuse and specular term. Fix: set the new vertices' tangent to `[1, 0, 0, 1]`, matching what `from_triangles_open` writes for its own vertices.

4.3 Layer 3: Zero normals, masked by a gradient overwrite

A second invisible factor was that *all* clip-vertex normals were `[0, 0, 0]`. We thought `overwrite_normals_with_grad` repaired them, but the fidget grad evaluator returned zero gradients for every point we tried; the function's `if len > 1e-10` guard then skipped the overwrite, leaving the zeros in place. For the original gyroid mesh this was masked by the prior smooth-normal pass. For clip vertices it was not.

The two-line fix was to call `recompute_smooth_normals(&mut mesh)` *after* the clip — a pure mesh-geometry computation that cannot fail in this way.

4.4 Pipeline of the fix

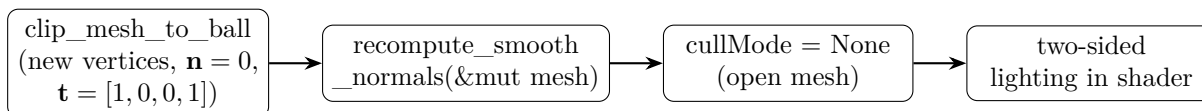


Figure 2: Four pieces had to align for the clipped patch to be lit correctly: non-zero tangents, geometry-based normals, no back-face culling, and two-sided lighting. Removing any one yields a dark mesh.

5 Lessons Learned

- For open implicit-surface patches, prefer *mesh-level* clipping over Boolean SDF combinations. The latter changes topology from 2-manifold to 3-manifold boundary and introduces spurious cap regions.
- Never rely on a numerical gradient as the sole source of vertex normals: a silent zero-gradient evaluation degrades to “no normals” rather than an error.
- Watch for NaN-propagation paths through `normalize(...)` — a single zero-length vector in a TBN frame can blacken every fragment using that vertex.

- Two-sided lighting (back-normal flip) is the default for any open mesh whose triangles can present either face to the camera.

6 Implementation Pointers

- `src/main.rs`
 - `clip_mesh_to_ball(mesh, c, r)` — exact spherical cutting of triangles.
 - `recompute_smooth_normals(&mut mesh)` — area-weighted normals from triangle geometry alone.
 - In `build_mesh`: after MC33/DC and the clip, call `recompute_smooth_normals`.
 - Clip-vertex initialiser uses `tangent = [1.0, 0.0, 0.0, 1.0]`.
- `crates/engvis-renderer/src/material_pipeline.rs`
 - Solid pipeline: `cullMode = None`.
 - Fragment shader: `if dot(N,V) < 0.0 { N = -N; }` before lighting.